Project 1 - KVDB

You are tasked with writing a small database that will operate on numeric values. This database will need to implement a number of features which are standard with other small databases.

You have been given scaffold code that will be the following folders.

- `kvdb` - Module, you will need to implement functionality within this project itself.
- `kvdb-test` - Node-Tests, test cases are present to test your class library and ensure you have implemented the logic as per the documentation.

The backing data structure of `KVDBObject` must be an `UnrolledLinkedList`, this is a data structure that is a mixture of a `Array` and `LinkedList`. Refer to the next section regarding requirements.

UnrolledLinkedList

Similar to a `LinkedList`, an `UnrolledLinkedList` is composed of the following.

- `UnrolledNode`, Like a linked list, a node has a reference to its **next** node, however, an unrolled linked list node will have up to k elements within it. This is considered the capacity.

$k > 1$ and each node will have the same **capacity**. You will need to ensure that the number of elements (**length**) inside the node does not exceed the capacity.

- `UnrolledLinkedList`, The container for the nodes themselves. It will have a link to the first and last node. The `UnrolledLinkedList`, will need to support the following methods.
- `get(int index): T`, Using the index, it will return an element from the data structure, the index is a logical index, it does not map directly to a node and/or entry within a node. The index is related to the total number of elements contained. This method should aim to be a $O(1)$ operation.

If $index < 0$ or $index > length$, the method should return `null`.

- `prepend(T element): void`, adds an element to the front of the list.
- `append(T element): void`, adds an element to the end of the list, it should be added to the last node. This should be an $O(1)$ operation.
- `set(int index, T element): void`, given a specific index, the operation will replace the element at that given position, with the element argument. This should aim to be an $O(1)$ operation.
- `remove(int index): T`, removes an element from the list. Using the index, removes the element from a node, if the node is now empty, the operation should remove the node and update the previous node to connect

to the one after it or `null`. It should return the removed value `T` or `null` when completed. This method should aim to be $O(n)$ operation or better.

- `size(): number`, retrieves the size of the list, it must return an integer value that outlines how many elements have been added to the list.

Restrictions

There are some restrictions to your implementation.

- For each key and its associated values, you should hold all keys within an **UnrolledLinkedList**, where each key has an association with an **UnrolledLinkedList** that holds the values. As a hint, Given a key that exists within the **UnrolledLinkedList**, this should also hold data that refers to the index in the **values** collection. However, it is
- You are permitted to use an `Array` to hold the `UnrolledLinkstLists` that contain the values.

When attempting this assessment, you should ensure that you build the `UnrolledLinkedList` first but also test your construction to ensure that it works. Write unit-tests for the operations listed above.

- You are restricted to only using a `Int32Array` for holding the keys and values. The `Map` and `Set` datastructures are disallowed for this assessment. You are allowed to use the `Array` collection type

As a hint, you are free to create your own data structures. Outside of `Sort`, it is indicated that you do not utilise any method that will trivialise components of the assessment. You may rebuild certain components if you feel that is worth your time.

- You will need to adhere to the API set out in the initial scaffold. You are free to create your own classes and methods that can be called by the scaffold code. However, changing the name, return type or parameters will prevent the test cases from working.
- You are not allowed to use `static` keyword in association with variables without approval from the instructor. Using the `static` keyword in conjunction with methods is reasonable.
- You cannot add any more fields within the `KVDBSwitchboard` class. Separate the data from the switchboard with your own type or via `KVDBObject`.
- `KVDBObject` must have a constructor with no parameters.
- You are disallowed from using GenAI or LLMs for generating any artifacts for this assessment.

Background Information

While it may seem scary that you are implementing a database, it is not going to be much different from other tasks you have tackled (It sounds scarier than it looks).

The assessment will be testing your knowledge on:

- Conditions (if and else)
- Loops
- Arrays (specifically integer arrays)
- Classes
- Inheritance and Interfaces
- File/IO
- Testing

However, most of the work is loops and array manipulation.

In relation to **time-complexity**, your solutions should be

Operations to Implement

Below are a list of operations that you will build along with a small example of how it works. These operations have been broken up into following sets.

- Query Operations - Get, Set, Remove, Append, Push, View
- Set Operations - Sort, PartialSort, Union, Intersect
- Database Operations - Commit, Checkout, Load, and Save

Important Note, the operations, and information below is a form of **pseudo-code**. When referring to command, it is referring the method/operation that reflects that.

Query Operations

- `ListKeys` - Lists all the keys within the database. If the database is empty, no keys will be returned If the database has one or more keys, the keys should be returned

Assuming keys: 1, 54 and 88 exist, the result return should be:

```
Command = "ListKeys"
Results = [1, 54, 88]
Success = true
```

If no keys exist, it will return the following.

```
Command = "ListKeys"
Results = []
Success = true
```

- `GetEntry` - Given a key as an argument, it should return the entries associated with that key.

Assuming keys 32 exists and has the following data associated [22, 89, 54, 10]. Using `GetEntry 32` should return a result that will be:

```
Command = "GetEntry"
Results = [22, 89, 54, 10]
RecordedKey = 32
Success = true
```

If the key does not exist, the result should be:

```
Command = "GetEntry"
Results = []
RecordedKey = null
Success = false
```

- `SetEntry` - Given a key and data, the data should be associated with the key given or a new key spot is added and the data is set to that key.

If key `45` did not exist yet, `SetEntry 45 [43, 99, 65, 11]` will result in: A new key entry (45 added) And if you were to call `GetEntry 45`, it will return `[43, 99, 65, 11]`

```
Command = "SetEntry"
Results = [43, 99, 65, 11]
RecordedKey = 45
Success = true
```

If the key `45` already existed and had the values `[22, 11, 33]` associated, those values will be dropped when `SetEntry 45 [43, 99, 65, 11]` is specified.

- `RemoveEntry` - Given a key, it will remove the entries and key from the list of keys.

If key `87` existed within the database, the values associated with it will be drop, along with the key itself.

- Calling `ListKeys` will produce an array that would not include `87` in it
- Calling `GetEntry 87` will produce an unsuccessful result.

If key `87` did not exist within the database, this will be marked as an unsuccessful query.

- `Push` - Given a key and data associated, it will add the entries given to the front of the entry.

Let's assume key `54` existed and had the entry `[22, 44, 12]`, if a `Push 54 [88, 77, 66]` was conducted, it will be changed the entry set to produce:

```
Results = [88, 77, 66, 22, 44, 12]
Command = "Push"
RecordedKey = 54
Success = true
```

If the key does not exist, it will operate like `SetEntry`.

- `Append` - Given a key and data associated, it will add the entries given to the end of the entry.

Let's assume key `54` existed and had the entry `[22, 44, 12]`, if a `Append 54 [88, 77, 66]` was conducted, it will be changed the entry set to produce:

```
Results = [22, 44, 12, 88, 77, 66]
Command = "Append"
RecordedKey = 54
Success = true
```

If the key does not exist, it will operate like `SetEntry`.

- `View` - Given a key and a range to view. Given a particular range, it will output the range for that view only.

If the command was called like so `View 22 2 4` and we assume key `22` is associated with the entry `[44, 55, 66, 77, 88, 99]`. The result will be:

```
Command = View
Results = [66, 77, 88]
RecordedKey = 22
Success = true
```

Assuming the same entry but we were to call `View 22 5 10`, this will be out of range, as we cannot access beyond the array boundaries. Similarly we cannot have a negative value nor can the `endIndex` be greater than the `startIndex`.

Set Operations

- `Sort` - This operation will sort the values of an entry.

Let's assume the key `22` exists and has the entry `[65, 22, 99, 11]`. Using `Sort` on `22` will change the entry to show `[11, 22, 65, 99]`.

If the key does not exist, it is considered a failed operation.

- `PartialSort` - Given a range, it will sort a segment of the entry.

Let's assume the key `22` exists and has the entry `[65, 22, 99, 11]`. Using `PartialSort 22 1 3` will change the entry to show `[65, 11, 22, 99]`. It will have ignored the first index.

If the key does not exist, it is considered a failed operation.

- `Union` - Given two keys and their entries, it will create a result which is the combination (excluding duplicates).

Let's assume the key `10` and `20` exist.

If `10` has the entry `[45, 33, 89]` and if `20` has the entry `[33, 61, 5]`

The union between `10` and `20` will be `[5, 33, 45, 61, 89]`. Only 1 count of `33` is permitted, not multiple.

- `Intersect` - Given two keys and their entries, it will create a result which is the overlapping values.

Let's assume the key `10` and `20` exist.

If `10` has the entry `[45, 33, 89, 21, 5]` and if `20` has the entry `[33, 61, 5]`

The intersection between `10` and `20` will be `[5, 33]`. We will only include one value from a set and not multiple.

Database Operations

- `Commit` - Given a database state, it will save the state into memory for it be usable at a later point in time.

Let's assume we have a database with the following state.

Keys = `[1, 4, 7, 10]`

Entries = (respective keys)

```
[
  [1, 2, 3],
  [5, 4, 3],
  [2, 1],
  [8]
]
```

Commits = `[]`

Creating a commit will create a copy of the database at that given point in time and the `Commits` entries will change to now have `0` there. If commit is triggered again the commit index will be incremented to `1`.

- `Checkout` - Given an index value, it will load the database state associated with that commit.

Let's assume the last database commit which is index 3 had the following state.

Keys = `[1, 4, 7, 10]`

Entries = (respective keys)

```
[
  [1, 2, 3],
  [5, 4, 3],
  [2, 1],
  [8]
]
```

Commits = `[0, 1, 2]`

Calling `Checkout 3` will return the state above.

- `Load` - Will load database object and replace the current database (any commits, keys+values and watchers will be dropped) if successfully loaded.

The `Load` command will be given a `path` which is a string that refers to the filesystem.

If the file cannot be loaded, the current database state is preserved.

Loading and saving the database will be a matter of interpreting a CSV file.

As a hint, use `Split` to separate the `,` and read on a line-by-line basis.

Database is in a CSV format Format: Key0,Value0,Value1,Value2,... Key1,Value0,Value1,Value2,...

Using an example file.

```
1, 1, 2, 3
4, 5, 4, 3
7, 2, 1
10, 8
```

It should construct a database with the following state.

Keys = `[1, 4, 7, 10]`

Entries = (respective keys)

```
[
  [1, 2, 3],
  [5, 4, 3],
  [2, 1],
  [8]
]
```

- `Save` - Will save the current database state to a CSV file.

You will need to perform some amount of string concatenation to ensure the values are recorded correctly.

Given the database state:

Keys = `[1, 4, 7, 10]`

Entries = (respective keys)

```
[
  [1, 2, 3],
  [5, 4, 3],
  [2, 1],
  [8]
]
```

The file that should be outputted should resemble the following.

```
1, 1, 2, 3
4, 5, 4, 3
7, 2, 1
10, 8
```

`Save` accepts a pathname which will also include the name of the file. As an example `Save MySampleDatabase.csv`.

Submission, Milestones, Testing, and Instructor Test Cases

You are encouraged to write your own test-cases and attempt to cover branches and cases not covered by the instructor's test cases. Specifically, you will need to test your unrolled linked list and provide a report on the runtimes of each function.

As a hint, the instructor test cases are likely to provide typical-scenarios/happy scenarios and not cases where bad-input is at play. Consider results which may not be well specified or may require you make assumptions. You can discuss your assumptions with the instructor to check for their validity.

Checklist

Use this as a checklist to ensure you have completed the assessment.

- Constructed a `KVDBObject` class with the necessary fields
- Implement an **Unrolled Linked List**
- Implement all **Query Operations**
- Implement all **Set Operations**
- Implement all **Database Operations**
- Construct test cases for some **Query Operations**
- Construct test cases for some **Set Operations**
- Construct test cases for some **Database Operations**
- Implementation passes the last release of the Instructor Test Cases